

Optimal Use of WIP Limits. Why is WIP Important? | by Michael Ryan | Medium

By Michael Ryan

Optimal Use of WIP Limits Why is WIP Important? Work-In-progress (WIP) is a concept that originated in the Lean manufacturing movement. In both a financial and operational sense WIP is considered a form of waste: work that has been done and paid for but is providing no benefit to the customer or the company. WIP is an even more insidious problem in a software development context. In addition to financial and operational complications WIP imposes an additional tax on a software engineering team in the form of increased complexity and cognitive load.

WIP in Manufacturing The ideal process in both manufacturing and software is often single-piece flow. In single-piece flow once work begins it continues without interruption until value is delivered to a customer. WIP is reduced to its theoretical minimal level with single-piece flow. On the other hand, consider what happens in the absence of single-piece flow. Some value-adding work has been done on a product — but then work has to stop for a while before it continues and the product is finished. Maybe required materials are not present and the product must be stored until those materials arrive. Perhaps the product needs to be moved to another location before value-adding work can begin again — the time and expense involved in moving the product is a form of waste. Partially finished items need to be kept track of, moved and stored until they can be completed and delivered to customers. The extra effort to do this provides no value to the customer or the company and is wasteful.

WIP in Software Development The issues with partially finished work are similar in software development. In this context WIP is source code or other resources that have been created but are not yet in production producing value for users. In most cases WIP takes the form of completed code that is not yet in production. It can be code on an engineer's laptop that is not committed to a repository, code committed to a repository but not yet in a DevOps pipeline, or code in a DevOps pipeline waiting to be deployed. Either way it is waste: something that time and money has been spent on that is not producing value for the customer. A software development team must spend precious cognitive bandwidth managing WIP. This takes effort and reduces the amount of time and energy that can be spent delivering features that provide value to the customer. In software development, as in manufacturing, it is very important to keep WIP at a minimum.

Cognitive load is a concept introduced in 1988 by psychologist John Sweller and recently made more popular by the book *Team Topologies: Organizing Business and Technology Teams for Fast Flow* by Matthew Skenton and Manuel Pais. In software engineering there are three main ways cognitive load or mental capacity can be spent: 1) needless tasks that provide limited value such as manual deployments or paperwork, 2) well-known foundational elements of the current task like knowledge of a particular programming language or commonly used libraries, or 3) higher level “value add” thinking about architectural concerns or code-level tactical decisions like API design. A team's capacity to handle cognitive load is very limited. Great care must be taken to ensure the most time is spent on higher level “value add” thinking. A high amount of WIP often means engineers are forced to work on more than one item at a time. Often a great many items at the same time. Paying attention to all these tasks increases the cognitive load on both individuals and the overall team while increasing the complexity of the tasks the team must handle. Excessive WIP can cause the performance and morale of a team to deteriorate badly. On the other hand, low WIP implies each engineer can focus on just a few (or ideally, one) thing at a time. The ideal state is when an engineer begins a task they have everything needed to complete that task and can work uninterrupted until that task is completed. There is often a small pause until that code passes code review, at which point it enters a build pipeline and again continues without interruption until it is released into production. This seamless and constant flow of code

reduces cognitive load on individuals and the overall team. Limiting WIP shortens the lead time for new features, reduces context switching, and allows engineers to concentrate resources and attention on the most important tasks. When engineers focus on only one thing at a time they are more productive and generally more satisfied with their work. They are more likely to stay with you. What happens if automated testing and deployment of code changes is not in place? Then that code needs to be stored somewhere (often long running release branches) while waiting for approval. The team needs to keep track of what features are in what branch and be aware of any interactions between branches. In practice, there are often quite a few interactions between long running branches which makes testing difficult. The context switching involved in this approach imposes a high cognitive load on a team and greatly reduces their capacity to provide value to the customer and company.

The Wrong Branching Model Can Increase WIP

Gitflow vs Trunk-based Development

Many software engineering teams use long-running release branches, sometimes multiple release branches. In this approach releases are infrequent and the idea is to work on multiple features at once and have them scheduled for multiple release dates. There may be valid business reasons this approach is used, and if that is the case then not much can be done. However, most often long-running release branches are in place simply because a team does not know how to deliver more frequent, smaller releases. By definition, each of these long-running release branches contains WIP. By definition each release branch is WIP. Engineers have to keep track of what features are in each release branch while also developing new features. This sort of structure creates distraction and requires frequent context switching. It can be very error-prone. Using the right branching strategy is critical to minimizing WIP. Branching models like Gitflow (<https://nvie.com/posts/a-successful-git-branching-model/>) split the code base into several long running branches, each designated for a future release. The cognitive load on a team using Gitflow tends to be high. They have to remember what each branch is for, what features are included in each branch, and how code in each branch interacts with other branches. There are many checks as code winds its way to production. Gitflow is the right tool for certain use cases, but the payment for all the protection it offers are very large, infrequent releases and large amounts of WIP.

Gitflow: image by Vincent Driessen

Trunk-based Development

image by Toptal To reduce WIP (and increase deployment frequency/decrease deployment size), take a look at Trunk-based development (<https://trunkbaseddevelopment.com/>). Trunk-based development is a practice where developers merge small, frequent updates to a core main branch. Trunk-based development has strong support in the DevOps community as it streamlines merging, reduces WIP and reduces cognitive load.

How WIP Limits are Used

Many teams try to use WIP Limits as a way of reducing WIP and the waste and high cognitive load a high level of WIP places on a team. By focusing on an increasingly smaller number of tasks (the theoretical ideal WIP Limit is 1), WIP Limits are used to reduce rework and duplicative efforts, eliminate the context switching that comes with juggling multiple tasks, cut down on unnecessary meetings, improve communications and eliminate handoffs between teams or parts of teams.

What is a WIP Limit?

A WIP Limit is simply a limit on the number of unfinished tasks an engineer or team can work on at the same time. The optimal WIP limit is 1 — meaning an engineer works on only one task until that task is completed. Once the task is completed the engineer switches to the next task. The goal is to minimize (and ultimately prevent) multitasking and the context switching that happens each time an engineer stops working on one task and picks up another. In many cases the rule is that if a WIP Limit is reached engineers perform non-coding tasks like code reviews, training, etc. Reducing WIP to a bare minimum is an important goal in any DevOps or other workflow transformation. However, WIP Limits need to be used very carefully or they can backfire. This article provides a quick background on the concept WIP Limits as well as advice on how to best use WIP Limits. As discussed, the optimal WIP Limit is one. In an ideal workflow, an engineer would work on one item until it is completed and passed off to an automated CI/CD pipeline

that would then test and deploy that code. The engineer would then immediately begin work on the next item with no delay. In this scenario the engineer begins the task with everything needed to complete their task without interruption (the exception often being a little time spent waiting for code reviews). Using WIP Limits for a Scrum Team The illustration below shows a simple and hypothetical development team scrum board. Stories in the left column are not started, the middle column contains stories that are in progress, and finally the column on the right where completed stories are placed. Example of team WIP limit of 4 tasks worked on at once. WIP Limits deal with items in the “In Progress” column with the goal of reducing the number of tasks being simultaneously worked. On some teams WIP Limits apply to the entire team, while other teams apply WIP Limits to each individual. In either case the objective is to reduce the amount of work-in-progress and all the negative effects discussed above. Where WIP Limits Work — and Where They Don't WIP Limits are often triggered when the entire engineering work stream is not set up for optimal flow. The typical symptom of non-optimal flow is when work on a story is partially completed, but then work has to stop while the engineer waits for clarification on requirements, waits for some approval to proceed further, or has to drop their work to focus on an important production defect. If there is a WIP Limit of 1 then that blocked engineer is idle — they can't work on anything else until the blockage is cleared. Idle engineers are obviously wasteful and few teams in this situation end up using WIP Limits for long, even if the idle time is used for training or other productive tasks. An increase in idle engineers often ends up being the fatal flaw in using WIP Limits to spur process improvements. Idle engineers are usually only tolerated if: A company is willing to let engineers be idle for possibly lengthy periods of time until the workflow is improved to the point where WIP Limits are not triggered. The engineering team using WIP Limits controls the parts of the process that need improvement. If the delays triggering a WIP Limit are the result of another department (possibly QA or Operations) then the fact your team has reached a WIP Limit just may not matter to them. It's possible these other groups might not be motivated to improve their pieces of the process so your team's WIP Limits are not triggered. Despite what anyone might say, most companies are not going to tolerate idle engineers for long. When the head of engineering complains their engineers are idle due to WIP Limits triggered because testing is not automated, the first question the group VP will ask is “Why can't the engineers work on something else while they wait?” Be careful how you answer this question. Arguing that WIP Limits are good because they reduce cognitive load and stress might not play well with stakeholders. One can always reduce cognitive load and stress simply by being idle. Instead the goal is to accomplish quite a bit while keeping cognitive load and stress in check. You don't want to be on the wrong side of this argument. Leading a Transformation If you are leading an agile or devops transformation (or the second, third or fourth attempt at a transformation) you need to be aware of organizational history, internal politics and manage how your team presents itself across the company. If past transformations have failed or only been partially successful the organization might be jaded on the prospect of any improvement. There is likely a long backlog of engineering work the business would have liked to be completed long ago. Stakeholders want engineers working to reduce that backlog. When stakeholders learn WIP Limits are reducing the amount of time engineers spend coding (even if only temporarily), precious political capital will need to be spent to keep them supporting the transformation. Why incur that expense? You'll need that political capital later, maybe if your team falls behind schedule or the first time you release a serious defect to production. Nothing ever goes as smoothly as you think it will. We have found it more effective and less politically risky to challenge a team to improve DORA devops metrics instead of implementing WIP Limits. Focusing on improving Deployment Frequency coupled with Change Failure Rate is often a good place to start. If your team is currently deploying once per month, challenge them to deploy every two weeks, then weekly, then at will. And do that without increasing the number of defects found in

production. From the DORA 2021 Report: Improving DORA metrics is a tangible task the team can focus on immediately. It will provide tangible benefits to the company. Tools like Value-Stream Mapping can be used to eliminate waste in any process. Remember, optimized processes minimize WIP. Accomplish just this and you'll be making a substantial contribution to the good of the company. The exception to this rule is if your team does not control the entire process. Maybe they are assigned to help another team by working on backlog items and they don't have any control or input into Deployment Frequency. In this case WIP Limits might be helpful, at least as a tool to help other teams understand they need to improve their processes. The Canary in the Coal Mine WIP Limits do have one great advantage. A triggered WIP Limit is often the first indication a high performing engineering team is beginning to backslide. If engineers start hitting WIP limits it means your DORA metrics are soon to decline and you have a problem to solve. A good approach is to set a WIP Limit just above what the normal WIP level is. In this sense WIP Limits are like the proverbial canary in the coal mine — an early warning system. To summarize, if you set a WIP Limit before flow is optimized you are bound to have engineers spend substantial time on non-engineering tasks. No one likes that. Engineers don't like that and your business stakeholders surely won't like that. You can argue the slack time produced by WIP Limits will be spent on other important tasks like training. That may be true but we think this is a bad way to manage important tasks. Your team will never know when the WIP Limit will be triggered and when they can spend time training, or when WIP will drop under the WIP Limit and they have to get back to engineering work. This seems like a haphazard approach to something like training, which is an important part of your team's professional development. Building Software that Thrives on Change In Kenzan's experience engineering organizations can be broken down into three groups: Base-level engineering organizations employ many manual processes. Performance declines rapidly under a limited amount of change or load. Their capacity for improvement is small and they can only recover to baseline performance with extreme difficulty. Effective engineering organizations produce good quality software while enduring a moderate amount of change or load. If stress exceeds a certain threshold performance rapidly declines. These organizations use some automated processes while still depending on manual work for important tasks. Their capacity for improvement is modest and they can recover to baseline performance with moderate difficulty. Elite engineering organizations grow stronger the more change or load is applied to them, in the same way the human body grows stronger through rigorous exercise. They actively seek to learn from mistakes and incorporate those learnings into future work, often embracing cloud infrastructure and cloud-native architectures. These organizations depend heavily on automation to increase quality while reducing cognitive load on teams and individuals. Their capacity for improvement is great and under stress they can recover to baseline performance with relative ease. Getting Agile and DevOps transformations right is critical to becoming an elite engineering organization. If you'd like to discuss the concepts included in this article, please reach out to the author at mryan@kenzan.com. Michael Ryan is Managing Principal Consultant and Head of Architecture at Kenzan. Author's linkedin page: <https://www.linkedin.com/in/michael-james-ryan/> If you'd like to discuss these concepts further please reach out to the author below: Michael Ryan Head of Architecture, Managing Principal Consultant mryan@kenzan.com